



High-Performance Computer Architectures

Practical Course

Single Instruction Multiple Data (SIMD) Instructions and FVec-Headers

Prof. Dr. Ivan Kisel

Robin Lakos

Akhil Mithran

Oddharak Tyagi

May 19, 2025

High-performance computer clusters are very expensive:

- locality (to avoid slowdowns by slow network communication)
- cooling (to keep it running over long periods)
- hardware
- energy

For smart cooling solutions and energy efficient clusters, there are ongoing researches. This goes hand in hand with the search for suitable buildings / installation sites (ideally at cool places).

Hardware, however, is slowly reaching its limits.

Hardware's Physical Limits

Moore's Law:

- by Gordon E. Moore
- every two years, the amount of
 - transistors,
 - resistors,
 - diodes and
 - capacitors is doubled.
- until today: quite accurate

Problem:

Core count is increasing over time,
frequency is slowly stagnating.

Nowadays: 5 nm technology (nearly on atomic level) with problems of

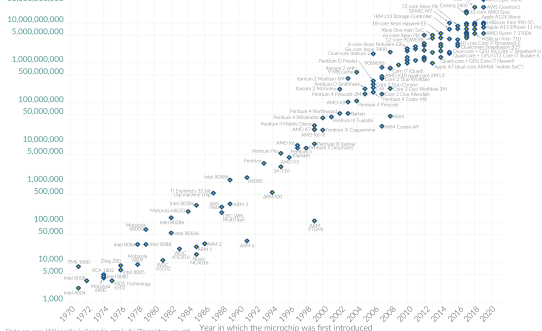
- quantum tunneling effects
- heat dissipation
- and others.

Moore's Law: The number of transistors on microchips doubles every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Transistor count

50,000,000,000



Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)

OurWorldInData.org - Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Our World
in Data

Therefore, to solve problems more efficiently, one can tackle two factors:

- frequency (has physical limits)
- parallel execution (has task specific dependency limits)

Frequency already reaches its physical limits → improve parallel execution.

More cores are available (see previous slide), but more hardware alone can not solve the problem. Idle cores do not solve any problems, they just generate costs.

The solution is to parallelize the tasks as good as possible and solve smaller independent sub-problems in parallel. Sometimes...

- ... entirely new algorithm ideas are required
- ... the computational effort increases (significantly)
- ... it is just not possible (due to strict data dependencies)

However, in high-performance computing, efficient algorithms are the key for success.

Classification of Multi-Processor Architectures

There are different concepts of parallelization, which were classified by Michael J. Flynn.

Flynn's Taxonomy:

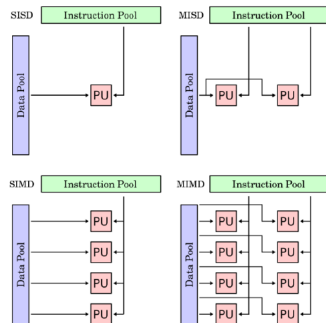
- Single Instruction Single Data (SISD)
- Single Instruction Multiple Data (SIMD)
- Multiple Instructions Single Data (MISD)
- Multiple Instructions Multiple Data (MIMD)

SISD is the sequential execution

SIMD is often referred to as "vectorization" (today's topic)

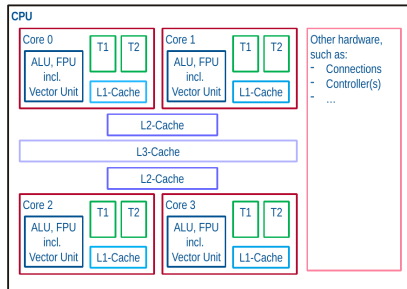
MISD is rarely used, but finds its niches (e.g. fault tolerance)

MIMD is typically implemented through multi-threading, multicore processors, and GPUs (discussed in a later topic)

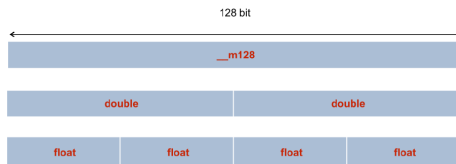


Parallelization with CPUs is possible by using

- vector units
 - in a core that supports SIMD instructions
- multiple threads handled on
 - single core (physical vs. logical cores)
 - multi-core
- multiple cores
 - in a single CPU
 - multiple CPUs (multi-socket mainboards) or
 - multiple machines (network)



SIMD-Register



- Register sizes usually measured in bits
- 128-, 256- and 512-Bit registers are typical

Data-Level Parallelism

How does SIMD work?

There are special functions / CPU instructions that allow to use the vector units of the CPU.

Example:

```
vc = add_vectors(va, vb); // 4 results at once
```

multiple data (4 values):

va[0] = 14.2f	va[1] = 0.11f	va[2] = 3.f	va[3] = -4.4f
---------------	---------------	-------------	---------------

single instruction (add):

+ + + +

multiple data (4 values):

vb[0] = 5.f	vb[1] = 1.f	vb[2] = -22.f	vb[3] = 3.f
-------------	-------------	---------------	-------------

multiple data (4 values):

vc[0] = 19.2f	vc[1] = 1.11f	vc[2] = -19.f	vc[3] = -1.4f
---------------	---------------	---------------	---------------

How do we get such a function `add_vectors`?

⇒ e.g. here: <https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#>

Arithmetic-, logical-, compare- and some extra (min, max, sqrt, inv. sqrt, ...) instructions available

Wrapper C++-Header for SIMD

- 4x32-Bit / 2x64-Bit (128 Bit) floating point SIMD vectors
- operator overloading (arithmetic, logic, compare, ...)
- function overloading (min, max, sqrt, fabs, ...)

```
class F32vec4
{
public:
    __m128 v;

    float & operator[](int i) { return (reinterpret_cast<float*>(&v))[i]; }
    float operator[](int i) const { return (reinterpret_cast<const float*>(&v))[i]; }

    F32vec4( __m128 v):v{v}{}
    F32vec4( const __m128 &a ):v(a) {}
    F32vec4( const float &a ):v(_mm_set_ps1(a)) {}

    F32vec4( const float &f0, const float &f1, const float &f2, const float &f3 ):v(_mm_set_ps(f3,f2,f1,f0)) {}

    /* Conversion function */
    operator __m128() const { return v; } /* Convert to __m128 */

    /* Arithmetic Operators */
    friend F32vec4 operator +(const F32vec4 &a, const F32vec4 &b) { return _mm_add_ps(a,b); }
    friend F32vec4 operator -(const F32vec4 &a, const F32vec4 &b) { return _mm_sub_ps(a,b); }
    friend F32vec4 operator *(const F32vec4 &a, const F32vec4 &b) { return _mm_mul_ps(a,b); }
    friend F32vec4 operator /(const F32vec4 &a, const F32vec4 &b) { return _mm_div_ps(a,b); }
    ...
}
```



```
1 #include "vectors/P4_F32vec4.h"
2 #include <iostream>
3
4 int main() {
5     fvec vc = fvec(14.2f, 0.11f, 3f, -4.4f) + fvec(5.f, 1.f, -22.f, 3.f);
6     std::cout << vc << std::endl;
7
8     return 0;
9 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
hpc@hpc-vm:~/Desktop/Exercise$ g++ simd_example.cpp -o simd_example.out ./simd_example.out
[19.2 1.11 -19 -3.4]
```

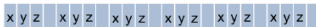
```
fvec vc = fvec(14.2f, 0.11f, 3f, -4.4f) + fvec(5.f, 1.f, -22.f, 3.f);
```

Back to Structs: AoS, SoA and AoSoA

Array of Structures (AoS)

```
struct TDataElement {  
    float x, y, z;  
};
```

```
TDataElement data[8];
```



- Vectorization needs data movement
- Padding wastes the memory
- + x, y, z in the same cache line
- + Intuitively structured code



Structure of Arrays (SoA)

```
struct TData {  
    float x[8], y[8], z[8];  
};
```

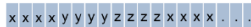
```
TData data;
```



- + The data is already grouped for vectorization
- + Compact data placement
- x, y, z in three different cache lines
- Confusing code



Array of Structures of Arrays



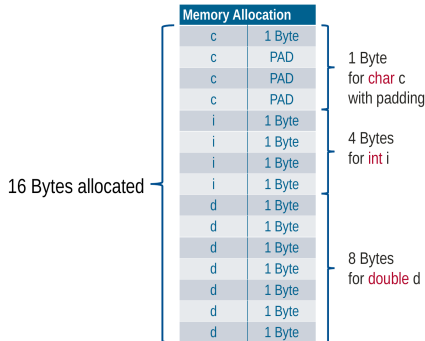
```
struct TDataVecElement {  
    float x[vecLen], y[vecLen], z[vecLen];  
};
```

```
TDataVecElement data[2];
```

Memory allocation is based on the largest data type

```
1 struct data {  
2     char c;  
3     int i;  
4     double d;  
5 };
```

- Size of data types:
 - char (8-Bit = 1 Byte)
 - int (32-Bit = 4 Bytes)
 - double (64-Bit = 8 Bytes)
- Multiples of 8-Byte are allocated and filled



Reinterpreting Memory (1)

Let us have a look at the effect of `reinterpret_cast`:

```
int a = 10;
std::cout<<"a = "<<a;      =>      a = 10
Memory: @0x7fe8eefaa010 => 000 ... 1010 (32bit)
```

```
float b = static_cast<float>(a);
std::cout<<"b = "<<b;      =>      b = 10.0000
Memory: @0x7fe8eefaa028 => 011 ... 0110 (32bit)
```

- Type conversion
- Data conversion
- Data copying
- Time consuming

```
float* c = reinterpret_cast<float*>(&a);
std::cout<<"*c = "<<*c;    =>      *c = 1.4013e-44
Memory: @0x7fe8eefaa010 => 000 ... 1010 (32bit)
```

- Type conversion
- No data conversion
- No data copying
- Fast

The problem occurs because the structure of the new data type is entirely different. Bits of an integer are stored in two's complement format, whereas bits of a float are stored in IEEE-754 format.

The new interpretation of the bits changes the meaning of the underlying data!

Reinterpreting Memory (2)

But instead of reinterpreting memory as a different data type that results in a different meaning, one can apply `reinterpret_cast` in situations, where the meaning does not change.

We can interpret 4 float values, stored consecutive (e.g. in an array) as an SIMD vector of 4 float values. The new data type just "collects" elements of the same data type.

```
float* c = reinterpret_cast<float*>(&a);
```

```
std::cout<<"*c = "<<*c;    =>    *c = 1.4013e-44
```

Memory: @0x7fe8eefaa010 => 000 ... 1010 (32bit)

- Type conversion
- No data conversion
- No data copying
- Fast

```
float scalar_arr[4];    float array
```

```
__m128 simd_vec;        SIMD vector stores 4 float
```

```
__m128& simd_vec = reinterpret_cast< __m128&>(scalar_arr[0]);
```

- Correct type conversion
- Data conversion is no needed
- No data copying
- Fast

Memory Alignment

Reinterpretation of memory is a nice but tricky tool - be careful.

All type conversions are allowed: there is no type-check by the compiler! Furthermore, non-aligned reinterpretation can lead to segmentation faults.

Not correctly aligned:

```
[byte 0] [byte 1] [byte 2] [byte 3] [byte 4] [byte 5] [byte 6] [byte 7] ...
          [ want_to_read ]
read > [ machine_word_1 ] [ machine_word_2 ]

+ reinterpret_cast<bigger type size>
= Segmentation fault
```

Solution: correct memory alignment

```
TYPE* arr = (TYPE*) __mm__malloc(sizeof(TYPE) * SIZE, ALIGNMENT)
```

Correctly aligned:

```
[byte 0] [byte 1] [byte 2] [byte 3] [byte 4] [byte 5] [byte 6] [byte 7] ...
[ want_to_read ]
read > [ machine_word ] ...
```

Example: Memory Alignment

128-Bit float fvec has a size of $4 \cdot 4 = 16$ bytes $\rightarrow$ alignment: 16 bytes.

- first element's memory address is dividable by 16
- pointer now points to fvec object, including all 4 float values

```
#include "vectors/P4_F32vec4.h"
```

```
#include <iostream>
```

```
#include <x86intrin.h>
```

```
int main() {  
    float* arr = (float*) _mm_malloc(sizeof(float) * 4, 16);  
    arr[0] = 1.f; arr[1] = 2.f; arr[2] = 3.f; arr[3] = 4.f;  
    fvec* vec = reinterpret_cast<fvec*>(arr);  
    return 0;  
}
```

The same works for references as well.